Facebook Data Analysis Project

Final Submission Version (Colab_Optimized)

This notebook coveres all aspects of the below mentioned pointers :

1. Data Access (.csv vs .xlsx comparison)
2. Data Examination (Excel file used for analysis)
3. Descriptive Statistics
4. Data Visualization (Controlled & Lightweight)

After running all cells: File → Download → Download .html

```
# STEP 1: Upload the two dataset files when prompted
from google.colab import files
uploaded = files.upload()
```

Choose Files Facebook Friends.csv
Facebook Friends.csv(text/csv) - 44028 bytes, last modified: 21/2/2026 - 100% done
Saving Facebook Friends.csv to Facebook Friends (2).csv

```
# STEP 2: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
```

1) Data Access

Load CSV and Excel files and compare.

```
## **Data Format and Structure** :
##Different sources often come in different formats (e.g., CSV, Excel, JSON, databases, APIs).
##Each format has its own way of structuring data, which affects how you read, parse, and process it.
 ##As you saw in this notebook, initially the CSV and Excel files presented different column headers,
 ##even though the underlying data content was similar once properly interpreted.


## **Data Quality and Integrity** :
 ##The source can greatly influence the quality of your data. Data from a well-maintained database might be clean and consisten
 ##The data scraped from a website or manually entered could be messy, incomplete, or contain errors.
 ##This impacts the reliability of your analysis.


## ** Metadata and Context** : Some data sources provide rich metadata (information about the data, like units, definitions, or
##which is crucial for correct interpretation.
 ##Other sources might lack this context, making the data harder to understand.


## **Accessibility and Licensing** : Where data comes from can affect its accessibility (e.g., public vs. private,
## requiring APIs or specific connectors) and any licensing restrictions on its use.


## **Performance** : Loading and processing data from very large databases or distributed systems
## will have different performance characteristics compared to local files.
##So, while the core analytical techniques might be similar, the initial steps of data access, cleaning, and preparation
##can vary widely and be quite complex depending on the data's origin.

##  Code begins !!

df_csv = pd.read_csv("Facebook Friends.csv")
df_excel = pd.read_excel("Facebook Friends.xlsx")
```

```
print("CSV Shape:", df_csv.shape)
print("Excel Shape:", df_excel.shape)

print("\nCSV Columns:")
print(df_csv.columns)

print("\nExcel Columns:")
print(df_excel.columns)
```

```
CSV Shape: (715, 3)
Excel Shape: (715, 18)

CSV Columns:
Index(['Friend \tAge\tPhotos\t# of Tags\tAlbums\tGender \tEmp\tProfile\tCover\tOrientation\tRelationship\tPosts\tReplies\tMidWes
       'Unnamed: 1', 'Unnamed: 2'],
      dtype='object')

Excel Columns:
Index(['Sample ', 'Age', 'Photos', '# of Tags', 'Albums', 'Gender ', 'Emp',
       'Profile', 'Cover', 'Orientation', 'Relationship', 'Posts', 'Replies',
       'Children', 'Likes', 'Edu', 'Events', 'Friends'],
      dtype='object')
```

Conclusion:
When structured properly, both file types load the same data.
Excel may preserve formatting more reliably, but content remains consistent.

2) Data Examination (Using Excel DataFrame)

```
df = df_excel.copy()

print("Column Names:")
print(df.columns)

print("\nTop 5 Rows:")
display(df.head())

print("\nBottom 5 Rows:")
display(df.tail())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Column Names:
Index(['Sample ', 'Age', 'Photos', '# of Tags', 'Albums', 'Gender ', 'Emp',
       'Profile', 'Cover', 'Orientation', 'Relationship', 'Posts', 'Replies',
       'Children', 'Likes', 'Edu', 'Events', 'Friends'],
      dtype='object')

Top 5 Rows:
```

	Sample	Age	Photos	# of Tags	Albums	Gender	Emp	Profile	Cover	Orientation	Relationship	Posts	Replies	Children	Likes	E
0	1	57	7	27	5	1	1	1	0	1	1	1	0	3	21	
1	2	42	531	241	19	0	0	0	0	1	1	5	1	3	9	
2	3	42	1396	423	40	0	1	1	0	1	1	186	31	2	43	
3	4	55	394	139	48	0	1	1	1	1	1	6	4	2	187	
4	5	30	916	231	78	1	1	1	1	1	1	412	752	1	34	

```
Bottom 5 Rows:
```

	Sample	Age	Photos	# of Tags	Albums	Gender	Emp	Profile	Cover	Orientation	Relationship	Posts	Replies	Children	Likes
710	711	25	499	4	0	0	1	1	1	1	1	16	19	0	49
711	712	22	232	101	4	1	0	1	0	1	0	698	552	0	223
712	713	22	327	329	29	1	1	1	0	1	1	543	399	0	520
713	714	24	219	0	8	1	1	1	1	1	0	2	7	1	66
714	715	19	465	0	22	0	1	1	0	1	1	3	6	0	990

```
Data Types:
Sample          int64
Age             int64
Photos          int64
# of Tags        int64
Albums          int64
Gender          int64
Emp             int64
Profile         int64
Cover           int64
Orientation     int64
Relationship    int64
Posts           int64
Replies         int64
Children        int64
Likes           int64
Edu             int64
Events          int64
Friends         int64
dtype: object

Missing Values:
Sample          0
Age             0
Photos          0
# of Tags       0
Albums          0
Gender          0
Emp             0
Profile         0
Cover           0
Orientation     0
Relationship    0
Posts           0
Replies         0
Children        0
Likes           0
Edu             0
Events          0
Friends         0
dtype: int64
```

3) Descriptive Statistics

```
## Descriptive statistics summarizes the main features of a dataset.
##They provide simple summaries about the sample and the measures,
##such as the count, mean, standard deviation, minimum, quartiles, and maximum values for each column,
 ##as we've seen from the df.describe() output. These measures help to understand the central tendency,
 ##dispersion, and shape of the data's distribution without making inferences beyond the current dataset.
```

```
stats = df.describe()
display(stats)
```

	Sample	Age	Photos	# of Tags	Albums	Gender	Emp	Profile	Cover	Orientation
count	715.000000	715.000000	715.000000	715.000000	715.000000	715.000000	715.000000	715.000000	715.000000	715.000000
mean	358.000000	24.406993	723.914685	458.096503	18.952448	0.541259	0.745455	0.917483	0.507692	0.956643
std	206.547008	6.969578	1265.112304	720.835481	22.502860	0.517932	0.435910	0.275344	0.500291	0.203801
min	1.000000	13.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	179.500000	21.000000	124.500000	75.500000	6.000000	0.000000	0.000000	1.000000	0.000000	1.000000
50%	358.000000	22.000000	320.000000	250.000000	12.000000	1.000000	1.000000	1.000000	1.000000	1.000000
75%	536.500000	25.000000	757.000000	566.500000	22.500000	1.000000	1.000000	1.000000	1.000000	1.000000
max	715.000000	81.000000	11995.000000	10460.000000	172.000000	2.000000	1.000000	1.000000	1.000000	1.000000

Next steps: Generate code with stats New interactive sheet

The key statistical attributes indicated by the descriptive statistics are:

The describe() function provides:

- Count
- Mean
- Standard deviation
- Minimum
- Quartiles (25%, 50%, 75%)
- Maximum

These indicate central tendency and dispersion.

Count : The count value, as shown in the output of df.describ(), represents the number of non-null (or non-missing) observations for each column. It tells us how many valid entries there are in each series.

Mean : The mean is the average value of the data points in a column. It is calculated by summing all the values in a column and then dividing by the number of values. It's a measure of central tendency.

Standard Deviation : The Standard Deviation measures the amount of variation or dispersion of a set of values. A low standard deviation indicates that the values tend to be close to the mean of the set, while a high standard deviation indicates that the values are spread out over a wider range.

Minimum : The Minimum value represents the smallest observation found in that column.

Inter Quartiles : The Interquartile Range (IQR) is the middle 50% of the data. It's the range between the first quartile (25th percentile) and the third quartile (75th percentile). It indicates the spread of the central portion of the data, and it's less sensitive to outliers than the full range.

Maximum : The maximum value represents the largest observation found in that column.

4) Data Visualization (Optimized)

Only first 4 numeric columns are plotted to ensure smooth execution.

```
## Descriptive statistics provide several key insights into your dataset:

## Central Tendency:
 ##Measures like the mean (average) give you an idea of the typical value of a variable.

## Dispersion (Spread): The standard deviation, minimum, maximum, and quartiles (25%, 50% also known as median, and 75%) help y
 ## A small standard deviation means data points are close to the mean, while a large one indicates a wider spread.

## Data Range: The minimum and maximum values show the full extent of your data.
## Distribution Shape: While not explicitly stated by each number, the combination of mean, median (50% quartile), and standard
## Outliers: The range and quartiles can also help identify potential outliers—data points that are unusually far from other ob
## Completeness: The count tells you if there are any missing values in a column, as it only counts non-null entries.
```

```
numeric_cols = df.select_dtypes(include='number').columns[:4]

# Histograms (limited)
for col in numeric_cols:
    plt.figure()
    df[col].hist()
    plt.title(f'Histogram of {col}')
    plt.show()
```